

A (Long) Peek into Reinforcement Learning

This PDF conversion (v1) was generated on October 10, 2025*

Online version: <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>.

Contents

1	A (Long) Peek into Reinforcement Learning	3
2	What is Reinforcement Learning?#	4
2.1	Key Concepts#	5
2.1.1	Model: Transition and Reward#	6
2.1.2	Policy#	6
2.1.3	Value Function#	6
2.1.4	Optimal Value and Policy#	7
2.2	Markov Decision Processes#	7
2.3	Bellman Equations#	8
2.3.1	Bellman Expectation Equations#	8
2.3.2	Bellman Optimality Equations#	9
3	Common Approaches#	9
3.1	Dynamic Programming#	9
3.1.1	Policy Evaluation#	9
3.1.2	Policy Improvement#	9
3.1.3	Policy Iteration#	10
3.2	Monte-Carlo Methods#	10
3.3	Temporal-Difference Learning#	11
3.3.1	Bootstrapping#	11
3.3.2	Value Estimation#	11
3.3.3	SARSA: On-Policy TD control#	12
3.3.4	Q-Learning: Off-policy TD control#	12
3.3.5	Deep Q-Network#	15
3.4	Combining TD and MC Learning#	18
3.5	Policy Gradient#	21
3.5.1	Policy Gradient Theorem#	22
3.5.2	REINFORCE#	23
3.5.3	Actor-Critic#	23
3.5.4	A3C#	24
3.6	Evolution Strategies#	25
4	Known Problems#	27
4.1	Exploration-Exploitation Dilemma#	27
4.2	Deadly Triad Issue#	28
5	Case Study: AlphaGo Zero#	28
6	References#	34
7	Citation	36
8	References	37

[Lil'Log](#)

- |
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 A (Long) Peek into Reinforcement Learning

Date: February 19, 2018 | Estimated Reading Time: 31 min | Author: Lilian Weng
Table of Contents

- [What is Reinforcement Learning?](#)
 - [Key Concepts](#)
 - * [Model: Transition and Reward](#)
 - * [Policy](#)
 - * [Value Function](#)
 - * [Optimal Value and Policy](#)
 - [Markov Decision Processes](#)
 - [Bellman Equations](#)
 - * [Bellman Expectation Equations](#)
 - * [Bellman Optimality Equations](#)
- [Common Approaches](#)
 - [Dynamic Programming](#)
 - * [Policy Evaluation](#)
 - * [Policy Improvement](#)
 - * [Policy Iteration](#)
 - [Monte-Carlo Methods](#)
 - [Temporal-Difference Learning](#)
 - * [Bootstrapping](#)
 - * [Value Estimation](#)
 - * [SARSA: On-Policy TD control](#)
 - * [Q-Learning: Off-policy TD control](#)
 - * [Deep Q-Network](#)
 - [Combining TD and MC Learning](#)

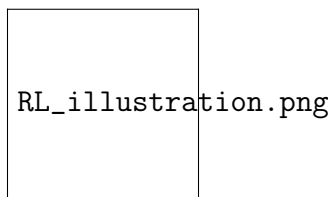


Figure 1: An agent interacts with the environment, trying to take smart actions to maximize cumulative rewards.

- [Policy Gradient](#)
 - * [Policy Gradient Theorem](#)
 - * [REINFORCE](#)
 - * [Actor-Critic](#)
 - * [A3C](#)
- [Evolution Strategies](#)
- [Known Problems](#)
 - [Exploration-Exploitation Dilemma](#)
 - [Deadly Triad Issue](#)
- [Case Study: AlphaGo Zero](#)
- [References](#)

[Updated on 2020-09-03: Updated the algorithm of [SARSA](#) and [Q-learning](#) so that the difference is more pronounced.

[Updated on 2021-09-19: Thanks to , we have this post in [Chinese](#)].

A couple of exciting news in Artificial Intelligence (AI) has just happened in recent years. AlphaGo defeated the best professional human player in the game of Go. Very soon the extended algorithm AlphaGo Zero beat AlphaGo by 100-0 without supervised learning on human knowledge. Top professional game players lost to the bot developed by OpenAI on DOTA2 1v1 competition. After knowing these, it is pretty hard not to be curious about the magic behind these algorithms — Reinforcement Learning (RL). I'm writing this post to briefly go over the field. We will first introduce several fundamental concepts and then dive into classic approaches to solving RL problems. Hopefully, this post could be a good starting point for newbies, bridging the future study on the cutting-edge research.

2 What is Reinforcement Learning?#

Say, we have an agent in an unknown environment and this agent can obtain some rewards by interacting with the environment. The agent ought to take actions so as to maximize cumulative rewards. In reality, the scenario could be a bot playing a game to achieve high scores, or a robot trying to complete physical tasks with physical items; and not just limited to these.

The goal of Reinforcement Learning (RL) is to learn a good strategy for the agent from experimental trials and relative simple feedback received. With the optimal strategy, the agent is capable to actively adapt to the environment to maximize future rewards.

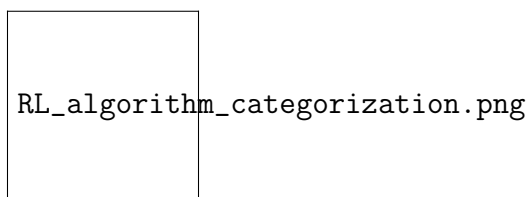


Figure 2: Summary of approaches in RL based on whether we want to model the value, policy, or the environment. (Image source: reproduced from David Silver's RL course [lecture 1](#).)

2.1 Key Concepts#

Now Let's formally define a set of key concepts in RL.

The agent is acting in an **environment**. How the environment reacts to certain actions is defined by a **model** which we may or may not know. The agent can stay in one of many **states** ($s \in \mathcal{S}$) of the environment, and choose to take one of many **actions** ($a \in \mathcal{A}$) to switch from one state to another. Which state the agent will arrive in is decided by transition probabilities between states (P). Once an action is taken, the environment delivers a **reward** ($r \in \mathcal{R}$) as feedback.

The model defines the reward function and transition probabilities. We may or may not know how the model works and this differentiate two circumstances:

- **Know the model:** planning with perfect information; do model-based RL. When we fully know the environment, we can find the optimal solution by [Dynamic Programming](#) (DP). Do you still remember “longest increasing subsequence” or “traveling salesmen problem” from your Algorithms 101 class? LOL. This is not the focus of this post though.
- **Does not know the model:** learning with incomplete information; do model-free RL or try to learn the model explicitly as part of the algorithm. Most of the following content serves the scenarios when the model is unknown.

The agent's **policy** $\pi(s)$ provides the guideline on what is the optimal action to take in a certain state with **the goal to maximize the total rewards**. Each state is associated with a **value** function $V(s)$ predicting the expected amount of future rewards we are able to receive in this state by acting the corresponding policy. In other words, the value function quantifies how good a state is. Both policy and value functions are what we try to learn in reinforcement learning.

The interaction between the agent and the environment involves a sequence of actions and observed rewards in time, $t=1, 2, \dots, T$. During the process, the agent accumulates the knowledge about the environment, learns the optimal policy, and makes decisions on which action to take next so as to efficiently learn the best policy. Let's label the state, action, and reward at time step t as S_t , A_t , and R_t , respectively. Thus the interaction sequence is fully described by one **episode** (also known as “trial” or “trajectory”) and the sequence ends at the terminal state S_T :

$S_1, A_1, R_1, S_2, A_2, \dots, S_T$

Terms you will encounter a lot when diving into different categories of RL algorithms:

- **Model-based:** Rely on the model of the environment; either the model is known or the algorithm learns it explicitly.
- **Model-free:** No dependency on the model during learning.
- **On-policy:** Use the deterministic outcomes or samples from the target policy to train the algorithm.
- **Off-policy:** Training on a distribution of transitions or episodes produced by a different behavior policy rather than that produced by the target policy.

2.1.1 Model: Transition and Reward#

The model is a descriptor of the environment. With the model, we can learn or infer how the environment would interact with and provide feedback to the agent. The model has two major parts, transition probability function P and reward function R .

Let's say when we are in state s , we decide to take action a to arrive in the next state s' and obtain reward r . This is known as one **transition** step, represented by a tuple (s, a, s', r) .

The transition function P records the probability of transitioning from state s to s' after taking action a while obtaining reward r . We use $\mathbb{P}$ as a symbol of "probability".

$$P(s', r \mid s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a]$$

Thus the state-transition function can be defined as a function of $P(s', r \mid s, a)$:

$$P_{ss'}^a = P(s' \mid s, a) = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} P(s', r \mid s, a)$$

The reward function R predicts the next reward triggered by one action:

$$R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} P(s', r \mid s, a)$$

2.1.2 Policy#

Policy, as the agent's behavior function π , tells us which action to take in state s . It is a mapping from state s to action a and can be either deterministic or stochastic:

- Deterministic: $\pi(s) = a$.
- Stochastic: $\pi(a \mid s) = \mathbb{P}[A=a \mid S=s]$.

2.1.3 Value Function#

Value function measures the goodness of a state or how rewarding a state or an action is by a prediction of future reward. The future reward, also known as **return**, is a total sum of discounted rewards going forward. Let's compute the return G_t starting from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discounting factor $\gamma \in [0, 1]$ penalize the rewards in the future, because:

- The future rewards may have higher uncertainty; i.e. stock market.
- The future rewards do not provide immediate benefits; i.e. As human beings, we might prefer to have fun today rather than 5 years later ;).
- Discounting provides mathematical convenience; i.e., we don't need to track future steps forever to compute return.
- We don't need to worry about the infinite loops in the state transition graph.

The **state-value** of a state s is the expected return if we are in this state at time t , $V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Similarly, we define the **action-value** (“Q-value”; Q as “Quality” I believe?) of a state-action pair as:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Additionally, since we follow the target policy π , we can make use of the probability distribution over possible actions and the Q -values to recover the state-value:

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) Q_{\pi}(s, a)$$

The difference between action-value and state-value is the action **advantage** function (“A-value”):

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

2.1.4 Optimal Value and Policy#

The optimal value function produces the maximum return:

$$V_*(s) = \max_{\pi} V_{\pi}(s), \quad Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

The optimal policy achieves optimal value functions:

$$\pi_* = \arg\max_{\pi} V_{\pi}(s), \quad \pi_* = \arg\max_{\pi} Q_{\pi}(s, a)$$

And of course, we have $V_{\pi_*}(s) = V_*(s)$ and $Q_{\pi_*}(s, a) = Q_*(s, a)$.

2.2 Markov Decision Processes#

In more formal terms, almost all the RL problems can be framed as **Markov Decision Processes** (MDPs). All states in MDP has “Markov” property, referring to the fact that the future only depends on the current state, not the history:

$$\mathbb{P}[S_{t+1} | S_t] = \mathbb{P}[S_{t+1} | S_1, \dots, S_t]$$

Or in other words, the future and the past are **conditionally independent** given the present, as the current state encapsulates all the statistics we need to decide the future.

A Markov decision process consists of five elements $\langle \mathcal{M} \rangle = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where the symbols carry the same meanings as key concepts in the [previous](#) section, well aligned with RL problem settings:

- $\mathcal{S}$ - a set of states;
- $\mathcal{A}$ - a set of actions;
- P - transition probability function;

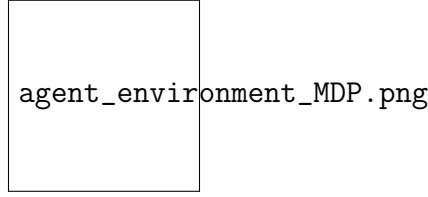


Figure 3: The agent-environment interaction in a Markov decision process. (Image source: Sec. 3.1 Sutton & Barto (2017).)

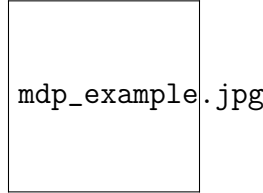


Figure 4: A fun example of Markov decision process: a typical work day. (Image source: randomant.net/reinforcement-learning-concepts)

- R - reward function;
- γ - discounting factor for future rewards. In an unknown environment, we do not have perfect knowledge about P and R .

2.3 Bellman Equations#

Bellman equations refer to a set of equations that decompose the value function into the immediate reward plus the discounted future values.

$$\begin{aligned} V(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s] \end{aligned}$$

Similarly for Q-value,

$$\begin{aligned} Q(s, a) &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi}[Q(S_{t+1}, a') \mid S_t = s, A_t = a]] \end{aligned}$$

2.3.1 Bellman Expectation Equations#

The recursive update process can be further decomposed to be equations built on both state-value and action-value functions. As we go further in future action steps, we extend V and Q alternatively by following the policy π .

$$\begin{aligned} V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a \mid s) Q_{\pi}(s, a) \\ Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \\ V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \big(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \big) \\ Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q_{\pi}(s', a') \end{aligned}$$

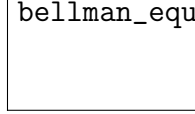

 bellman_equation.png

Figure 5: Illustration of how Bellman expectation equations update state-value and action-value functions.

2.3.2 Bellman Optimality Equations#

If we are only interested in the optimal values, rather than computing the expectation following a policy, we could jump right into the maximum returns during the alternative updates without using a policy. RECAP: the optimal values V_* and Q_* are the best returns we can obtain, defined [here](#).

$$\begin{aligned} V_*(s) &= \max_{a \in \mathcal{A}} Q_*(s, a) \\ Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \\ V_*(s) &= \max_{a \in \mathcal{A}} \big(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \big) \\ Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} Q_*(s', a') \end{aligned}$$

Unsurprisingly they look very similar to Bellman expectation equations.

If we have complete information of the environment, this turns into a planning problem, solvable by DP. Unfortunately, in most scenarios, we do not know $P_{ss'}^a$ or $R(s, a)$, so we cannot solve MDPs by directly applying Bellman equations, but it lays the theoretical foundation for many RL algorithms.

3 Common Approaches#

Now it is the time to go through the major approaches and classic algorithms for solving RL problems. In future posts, I plan to dive into each approach further.

3.1 Dynamic Programming#

When the model is fully known, following Bellman equations, we can use [Dynamic Programming](#) (DP) to iteratively evaluate value functions and improve policy.

3.1.1 Policy Evaluation#

Policy Evaluation is to compute the state-value V_π for a given policy π :

$$V_{t+1}(s) = \mathbb{E}_\pi [r + \gamma V_t(s') \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s', r} P(s', r \mid s, a) (r + \gamma V_t(s'))$$

3.1.2 Policy Improvement#

Based on the value functions, Policy Improvement generates a better policy $\pi' \geq \pi$ by acting greedily.

$$Q_\pi(s, a) = \mathbb{E} [R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a] = \sum_{s', r} P(s', r \mid s, a) (r + \gamma V_\pi(s'))$$

3.1.3 Policy Iteration#

The *Generalized Policy Iteration (GPI)* algorithm refers to an iterative procedure to improve the policy when combining policy evaluation and improvement.

$$\pi_0 \xrightarrow{\{\text{evaluation}\}} V_{\pi_0} \xrightarrow{\{\text{improve}\}} \pi_1 \xrightarrow{\{\text{evaluation}\}} V_{\pi_1} \xrightarrow{\{\text{improve}\}} \pi_2 \xrightarrow{\{\text{evaluation}\}} \dots \xrightarrow{\{\text{improve}\}} \pi^* \xrightarrow{\{\text{evaluation}\}} V_{\pi^*}$$

In GPI, the value function is approximated repeatedly to be closer to the true value of the current policy and in the meantime, the policy is improved repeatedly to approach optimality. This policy iteration process works and always converges to the optimality, but why this is the case?

Say, we have a policy π and then generate an improved version π' by greedily taking actions, $\pi'(s) = \arg\max_{a \in \mathcal{A}} Q_{\pi}(s, a)$. The value of this improved π' is guaranteed to be better because:

$$\begin{aligned} Q_{\pi}(s, \pi'(s)) &= Q_{\pi}(s, \arg\max_{a \in \mathcal{A}} Q_{\pi}(s, a)) \\ &= \max_{a \in \mathcal{A}} Q_{\pi}(s, a) \geq Q_{\pi}(s, \pi(s)) = V_{\pi}(s) \end{aligned}$$

3.2 Monte-Carlo Methods#

First, let's recall that $V(s) = \mathbb{E}[G_t \mid S_t = s]$. Monte-Carlo (MC) methods uses a simple idea: It learns from episodes of raw experience without modeling the environmental dynamics and computes the observed mean return as an approximation of the expected return. To compute the empirical return G_t , MC methods need to learn from **complete** episodes $S_1, A_1, R_2, \dots, S_T$ to compute $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ and all the episodes must eventually terminate.

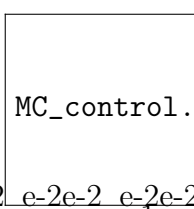
The empirical mean return for state s is:

$$V(s) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s]}$$

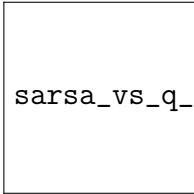
where $\mathbb{1}[S_t = s]$ is a binary indicator function. We may count the visit of state s every time so that there could exist multiple visits of one state in one episode ("every-visit"), or only count it the first time we encounter a state in one episode ("first-visit"). This way of approximation can be easily extended to action-value functions by counting (s, a) pair.

$$Q(s, a) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a]}$$

To learn the optimal policy by MC, we iterate it by following a similar idea to GPI.

				
1. 3.3 Temporal-Difference Learning Improve the policy greedily with respect to the current value function: $\pi(s)$	to Carlo Difference Learning (TD) Learn- ing is model-free and learns from episodes of $\arg\max_a \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t A_t]$ in $\mathbb{R}^{ A }$ space. However, TD learn- ing can learn from incomplete episodes and hence we don't need to track the episode up to termination.	idea in TD learn- ing is to up- date tar- gets with re- gard to ex- ist- ing es- ti- mates rather than ex- clu- sively re- ly- ing on ac- tual re- wards and com- plete re- turns as in MC meth- ods. This ap- proach is known as boot-	Es- ti- ma- tion# $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - Q(S_t, A_t))$ key# $V(S_t) \leftarrow V(S_t) + \alpha (R_t - V(S_t))$ value# $V(S_t) \leftarrow V(S_t) + \alpha (R_t - V(S_t))$	Next, $Q(S_t, A_t)$ dig into $Q(S_t, A_t)$ fun + part α on $(R_{t+1} + \gamma V(S_{t+1}) - Q(S_t, A_t))$ learn $Q(S_{t+1}, A_{t+1})$ mal pol- icy in TD learn- ing (aka "TD con- trol"). Be pre- pared, you are gonna see many fa- mous names of clas- sic al- go- rithms in this sec- tion.
2. Generate a new episode with the new pol- icy π (i.e. us- ing al- go- rithms like greedy helps us bal- ance	TD learn- ing can learn from incomplete episodes and hence we don't need to track the episode up to termination.	ly- ing on ac- tual re- wards and com- plete re- turns as in MC meth- ods. This ap- proach is known as boot-	$V(S_{t+1}) \leftarrow V(S_{t+1}) + \alpha (R_{t+1} - V(S_{t+1}))$ TD target). To what ex- tent we want to up- date the value func- tion is con- trolled	you are gonna see many fa- mous names of clas- sic al- go- rithms in this sec- tion.

e-2e-2	e-2e-2	e-2e-2	e-2e-2	e-2e-2
3.3.3	SARSA: On-Policy TD control#	1.	3.3.4	Q-Learning: Off-policy TD control#
	refers to the procedure of updating Q-value by following a sequence of $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots$. The idea follows the same route of GPI. Within one episode, it works as follows:	1. Initialize $\$t=0\$$.		1. Initialize $\$t=0\$$.
		2. Start we need to choose the <i>next</i> action according to the <i>current policy</i> . $\arg\max_a Q(S_0, a)$, where ϵ -greedy is commonly applied.		2. Starts with $\$S_0\$$.
		3. At time $\$t\$$, after applying action A_t , we observe reward R_{t+1} and get into the next state $\$S_{t+1}\$$.		3. At time step $\$t\$$, we pick the action according to Q values, $\$A_t = \arg\max_a Q(S_t, a)\$$ and ϵ -greedy is commonly applied.
				4. After applying action A_t , we observe re-


 sarsa_vs_q_learning.png

e-2e-2The

[t]2.3076923076923078e-2

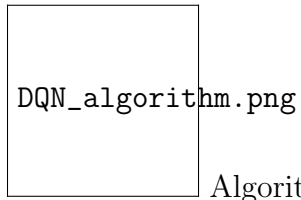
The backup diagrams

key
dif-
fer-
ence
from
SARSA
is
that
Q-
learning
does
not
fol-
low
the
cur-
rent
pol-
icy
to
pick
the
sec-
ond
ac-
tion
 A_{t+1} .

It
es-
ti-
mates
 Q^*
out
of
the
best
Q
val-
ues,
but
which
ac-
tion
(de-
noted
as
 a^*)
leads

for Q-learning and SARSA. (Image source: Replotted based on Figure 6.5 in Sutton &

Barto (2017))	e-2e-2	e-2e-2	The 3rd	Ulf	2012	DeeP	e-2e-2	The 2e-2	\$e-2e-2	where	In
			3.3.5	we	Q-	Q-	Experience		=	is	di-
			can	learning	Network	Re-	Re-				tion,
			mem-	may	(“DQN”;	play	on		$\mathbb{E}_{\pi}$	{(	stion,
			o-	suf-	Mnih	All	looks	a,	uni-	it	
			rize	fer	et	the	like	r,	form	is	
			$Q_*(\cdot)$	from	al.	episode	hide	s')	dis-	also	
			for	in-	2015)	steps		$\sim$	tri-	found	
			all	sta-	aims	S_t		$U(D)$	bu-	to	
			state-	bil-	to	=		$\Big[$	tion	be	
			action	ity	greatly	($S_t,$		$\big($	over	help-	
			pairs	and	im-	$A_t,$		$+$	the	ful	
			in	di-	prove	$R_t,$			re-	to	
			Q-	ver-	and	$S_{t+1})$		γ	clip		
			learning	gence	sta-	are		$\max_{a'} \pi$	them-	the	
			like	when	bi-	stored		$Q(s',$	ory	er-	
			in	com-	lize	in		$a';$	$D;$	ror	
			a	bined	the	one		θ^{π}	θ^{π}	term	
			gi-	with	train-	re-		$\}$	$\}$	to	
			gan-	an	ing	play		-	is	be	
			tic	non-	pro-	mem-			the	be-	
			ta-	lin-	ce-	ory		$Q(s,$	pa-	tween	
			ble.	ear	dure	D_t		$a;$	ram-	[-	
			How-	Q-	of	=		$\theta)$	e-	1,	
			ever,	value	Q-	$\{$		$\big)^2$	ters	1].	
			it	func-	learning	$e_1,$		$\Big]$	of	(I	
			quickly	tion	by	$\dots,$		$\$$	the	al-	
			be-	ap-	two	e_t			frozen	ways	
			comes	prox-	in-	$\}$		$\$$.	tar-	get	
			com-	i-	no-	D_t		$\$$	get	mixed	
			pu-	ma-	va-	has			Q-	feel-	
			ta-	tion	tive	ex-			networking		
			tion-	and	mech-	pe-				with	
			ally	boot-	a-	ri-				pa-	
			in-	strap-	nisms:	ence				ram-	
			fea-	ping		tu-				e-	
			si-	(See		ples				ter	
			ble	Prob-		over				clip-	
			when	lems		many				ping,	
			the	#2).		episodes.				as	
			state			Dur-				many	
			and			ing				stud-	
			ac-			Q-				ies	
			tion			learning				have	
			space			up-				shown	
			are			dates,				that	
			large.			sam-				it	
			Thus			ples				works	
			peo-			are				em-	
			ple			drawn				pir-	
			use			at				i-	
			func-			ran-				cally	
			tions			dom				but	



[t]2.3076923076923078e-2 Algorithm for DQN with experience replay and occasionally frozen optimization target. The preprocessed sequence is the output of some processes running on the input images of Atari games. Don't worry too much about it;

just consider them as input feature vectors. (Image source: Mnih et al. 2015)

e-2e-2There are many extensions of DQN to improve the original neural design, such as DQN with dueling architecture (Wang et al. 2016) which estimates state-value function $V(s)$ and advantage function $A(s, a)$ with shared network

TD_MC_DP_backups.png |

e-3e-3\$	e-3e-3This [t]9.473684210526316e-3	Comparison of the backup
----------	------------------------------------	--------------------------

<code>\begin{aligned}</code>	weighted
<code>\text{let}</code>	sum
<code>}</code>	of
<code>S</code>	many
<code>&=</code>	n-
<code>1</code>	step
<code>+</code>	re-
<code>\lambda</code>	turns
<code>+</code>	is
<code>\lambda^2</code>	called
<code>+</code>	-
<code>\dots</code>	return
<code>\\</code>	$G_t^{\{\lambda\}}$
<code>S</code>	=
<code>&=</code>	(1-
<code>1</code>	λ)
<code>+</code>	$\sum_{n=1}^{\infty}$
<code>\lambda(1</code>	$\lambda^{\{n-$
<code>+</code>	$1\}$
<code>\lambda</code>	$G_t^{\{n\}}\$.$
<code>+</code>	TD
<code>\lambda^2</code>	learn-
<code>+</code>	ing
<code>\dots)</code>	that
<code>\\</code>	adopts
<code>S</code>	-
<code>&=</code>	return
<code>1</code>	for
<code>+</code>	value
<code>\lambda</code>	up-
<code>S</code>	dat-
<code>\\</code>	ing
<code>S</code>	is
<code>&=</code>	la-
<code>1</code>	beled
<code>/</code>	as
<code>(1-</code>	TD() .
<code>\lambda)</code>	The
<code>\end{aligned}</code>	orig-
<code>\$\$</code>	i-

nal
ver-
sion
we
in-
tro-
duced
[above](#)

diagrams of Monte-Carlo, Temporal-Difference learning, and Dynamic Programming for state value functions. (Image source: David Silver's RL course [lecture 4](#): "Model-Free

Prediction”) e^{-3} e^{-3} All e^{-3} In e^{-3} $\$$ e^{-3} where e^{-3} Or e^{-3} $\$$ e^{-3} where e^{-3}

3.5 Policy dis- $\backslash\mathrm{mathcal}\{\mathcal{S}\}(\theta)$ $\backslash\mathrm{mathcal}\{\mathcal{A}\}(\pi)$

Gradient create = is con- = is

ent# space: $V_{\pi}(\theta)(S)$ in- $\sum_{s \text{ sta-}}$

we have $=$ ini- u- $\backslash\mathrm{in} \text{ tion-}$

in- $\backslash\mathrm{mathbb}\{\mathbb{E}\}_{\pi}[\backslash\mathrm{mathcal}\{\mathcal{S}\}]$

tro- $\$$ start- space: $d_{\pi}(\theta)(s)$

duced ing $V_{\pi}(\theta)(s)$

above state. = bu-

aim $\sum_{s \text{ tion}}$

to $\backslash\mathrm{in} \text{ of}$

learn $\backslash\mathrm{mathcal}\{\mathcal{S}\}$ $\backslash\mathrm{mathcal}\{\mathcal{A}\}$

the $\backslash\mathrm{Big}(\text{chain}$

state/ac- $d_{\pi}(\theta)(s)$

tion $\sum_{a \text{ } \pi}(\theta)$

value $\backslash\mathrm{in} \text{ If}$

func- $\backslash\mathrm{mathcal}\{\mathcal{A}\}$

tion $\pi(a \text{ are}$

and $\backslash\mathrm{vert} \text{ un-}$

then $s, \text{ fa-}$

to $\theta) \text{ mil-}$

se- $Q_{\pi}(s, \text{ iar}$

lect $a) \text{ with}$

ac- $\backslash\mathrm{Big}) \text{ the}$

tions $\$$ def-

ac- i-

cord- ni-

ingly. tion

Pol- of

icy a

Gra- “sta-

di- tion-

ent ary

meth- dis-

ods tri-

in- bu-

stead tion,”

learn please

the check

pol- this

icy [ref-](#)

di- [er-](#)

rectly [ence.](#)

with

a

pa-

ram-

e-

ter-

ized

func-

tion

Using 3.5.1 Policy Gradient Theorem

the gradient of the expected return with respect to the policy parameters θ is given by:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \pi(\cdot|\theta)} \left[\nabla_{\theta} \log \pi(a|s; \theta) \sum_{t=0}^{\infty} \gamma^t (R_t - V(s; \theta)) \right]$$

where $J(\theta)$ is the expected return, $\pi(a|s; \theta)$ is the policy, R_t is the reward at time t , and $V(s; \theta)$ is the value function.

is not differentiable (nice!), but surprisingly slow.

e-3e-3 This e-3e-3 REINFORCE e-3e-3

3.5.2 REINFORCE#

$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi(\theta)} [\nabla_{\theta} J(\theta)] \end{aligned}$

is as used

$\sum_s \text{named } \mathbb{E}_{\pi(\theta)} [\nabla_{\theta} J(\theta)]$

in “Pol- Carlo a-

$\nabla_{\theta} J(\theta)$ in pol- tion

d(s) Gra- icy of

$\sum_{a \in \mathcal{A}(s)} \nabla_{\theta} Q_{\pi}(s, a)$

in ent s, di- IN-

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} Q_{\pi}(s, a)$ ent, FORCE

$\pi(a|s)$ o- re- is

$\nabla_{\theta} J(\theta)$ rem” a)] lies to

s; which $\mathbb{E}_{\pi(\theta)}$ on sub-

$\nabla_{\theta} J(\theta)$ lays $\mathbb{E}_{\pi(\theta)} [\nabla_{\theta} Q_{\pi}(s, a)]$

$Q_{\pi}(s, a)$ the- a

$\nabla_{\theta} J(\theta)$ o- es- line

$\nabla_{\theta} J(\theta)$ ret- ti- value

$\nabla_{\theta} J(\theta)$ cal re- the

$\sum_s$ foun- turn re-

in da- by turn

$\nabla_{\theta} J(\theta)$ $\mathbb{E}_{\pi(\theta)}$ G_t

d(s) for meth- to

$\sum_{a \in \mathcal{A}(s)} \nabla_{\theta} Q_{\pi}(s, a)$ ods re-

in i- us- duce

$\nabla_{\theta} J(\theta)$ $\mathbb{E}_{\pi(\theta)}$ ing the

$\nabla_{\theta} J(\theta)$ pol- episode vari-

$\nabla_{\theta} J(\theta)$ icy sam- ance

$\nabla_{\theta} J(\theta)$ gra- ples, of

s; di- to gra-

$\nabla_{\theta} J(\theta)$ ent up- di-

$Q_{\pi}(s, a)$ al- date ent

$\nabla_{\theta} J(\theta)$ go- the es-

$\nabla_{\theta} J(\theta)$ rithms: pol- ti-

$\mathbb{E}_{\pi(\theta)}$ ma-

$\sum_s$ pa- tion

in ram- while

$\nabla_{\theta} J(\theta)$ e- keep-

d(s) ter ing

$\sum_{a \in \mathcal{A}(s)} \nabla_{\theta} Q_{\pi}(s, a)$ $\nabla_{\theta} J(\theta)$ the

in bias

$\nabla_{\theta} J(\theta)$ un-

$\pi(a|s)$ changed.

$\nabla_{\theta} J(\theta)$ For

$\nabla_{\theta} J(\theta)$ ex-

s; $\nabla_{\theta} J(\theta)$ am-

$\nabla_{\theta} J(\theta)$ ple,

$\nabla_{\theta} J(\theta)$ a

$\nabla_{\theta} J(\theta)$ com-

s; $\nabla_{\theta} J(\theta)$ mon

$\nabla_{\theta} J(\theta)$ base-

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

$\nabla_{\theta} J(\theta)$ $\nabla_{\theta} J(\theta)$

e-3e-3 If e-3e-3

3.5.3 Actor-Critic#

1. Initialize the value

at random

2. Generate in

one episode

$S_1, A_1, R_2, S_2, A_2, \dots, S_T$

3. For $t=1, 2, \dots, T$:

(a) Estimate

the return G_t

since the time step t .

(b) $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

•

Critic:

up-

dates

value

func-

tion

tion

pa-

ram-

e-

ters

w

and

de-

pend-

ing

on

the

al-

go-

ri-

thm

it

could

be

action-

value

$Q(a|s)$

$\nabla_{\theta} J(\theta)$

s;

w

or

state-

value

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

$V(s; w)$

e-3e-3Let's3e-3	e-3e-3 α 3e-3 β 3e-3 θ 3e-3	3.5.4 A3C#	A3C#	The3e-3Here3e-3	e-3e-3A3C
see	1. Initialize	and	van-	loss is	1. We en-
how	α , β	are	tage	the func-	ables
it	, two	'	Actor-	crit-	have the
works	w learn-	at	Critic	ics for	global par-
in	ing	ran-	(Mnih	learn state-	al-ram-
an	rates	dom;	et	the value	lelism
action-	for	dom;	al.,	is to	in
value	sam-	ple	2016),	min-	ters, mul-
actor-	ple	α	short	i-	and ti-
critic	and	β	for	mize	w; ple
al-	π	π	A3C,	the	sim- agent
go-	π	π	is	mean	i- train-
rithm.	π	π	a	squared	lar ing.
	s;	π	clas-	er-	The
	θ	θ	sic	ror,	specific
			pol-	$\mathcal{J}_v$	pa- di-
	2. For		icy	(w)	ram- ent
	t up-		gra-	=	e- ac-
	= dates,		di-	(G	ters, cu-
	1. . re-		ent	-	and mu-
	T: spec-		method	V(s;	la-
	tively.		with	w))^2\$	w'. tion
	(a)		the	and	2. step
	Sample		spe-	we	(6.2)
	re-		cial	use	Initialize
	ward		fo-	gra-	the can
	r_t		cus	di-	time be
	$\sim$		on	ent	step con-
	$R(s,$		par-	de-	t sid-
	a)\$		al-	scnt	= ered
	and		lel	to	1 as
	next		train-	find	a
	state		ing.	from	3. ref-
	s'			the	While
	$\sim$			op-	T ma-
	$P(s'$			ti-	<=tion
	$\sim$			mal	T MAX:
	s,			w.	minibatch-
	a)\$.			This	(a) based
				state-	Reset
				value	stochas-
				func-	tic gra-
				tion	di-
				is	gradient:
				used	d
				as	up=
				the	0:
				base-	the and
				line	dw
				in	ues
				the	of 0.
				pol-	(b)w
					or Synchronize
					get thread-

e-3e-3	e-3e-3	e-3e-3	e-3e-3	e-3e-3	e-3e-3
3.6	Evolution	Say,	Very	\$\$	We
State-	Strategies	we	dif-	$\begin{aligned}$	can
gies#	(ES)	start	fer-	$\&$	rewrite
	is	with	ent	∇_{θ}	this
	a	a	from	$\mathbb{E}_{\theta}$	θ
	type	pop-	the	$\sim$	mula
	of	u-	pop-	$N(\mu,$	in
	model-	la-	u-	$\sigma^2)$	terms
	agnostic	tion	lar	$F(\theta)$	of
	op-	of	MDP-	$\backslash\backslash$	a
	ti-	ran-	based	$=\&$	“mean”
	miza-	dom	ap-	∇_{θ}	pa-
	tion	so-	proaches	$\int_{\theta}$	ram-
	ap-	lu-	as	$F(\theta)$	e-
	proach.	tions.	what	$\Pr(\theta)$	ter
	It	All	we	$\&\&$	θ
	learns	of	have	$\text{Pr}(\cdot)$	(dif-
	the	them	in-	is	fer-
	op-	are	tro-	the	ent
	ti-	ca-	duced	Gaus-	from
	mal	pa-	above,	sian	the
	so-	ble	ES	den-	θ
	lu-	of	aims	sity	above;
	tion	in-	to	func-	this
	by	ter-	learn	tion.}	θ
	im-	act-	the	$\backslash\backslash$	is
	i-	ing	pol-	$=\&$	the
	tat-	with	icy	$\int_{\theta}$	base
	ing	the	pa-	$F(\theta)$	gene
	Dar-	en-	ram-	$\Pr(\theta)$	for
	win’s	vi-	e-	$\frac{\nabla_{\theta}}{\Pr(\theta)}$	θ
	the-	ron-	ter	$\Pr(\theta)$	$\Pr(\theta)$
	ory	ment	θ	$\backslash\backslash$	mu-
	of	and	with-	$=\&$	ta-
	the	only	out	$\int_{\theta}$	tion),
	evo-	can-	value	$F(\theta)$	ϵ
	lu-	di-	ap-	$\Pr(\theta)$	$\sim$
	tion	dates	prox-	∇_{θ}	$N(0,$
	of	with	i-	$\log$	$I)$
	species	high	ma-	$\Pr(\theta)$	and
	by	fit-	tion.	$\backslash\backslash$	there-
	nat-	ness	Let’s	$=\&$	fore
	u-	scores	as-	$\mathbb{E}_{\theta}$	θ
	ral	can	sume	$\sim$	$+$
	se-	sur-	the	$N(\mu,$	ϵ
	lec-	vive	dis-	$\sigma^2)$	σ
	tion.	(only	tri-	$[F(\theta)$	$\sim$
	Two	the	bu-	∇_{θ}	$N(\theta,$
	pre-	fittest	tion	$\log$	$\sigma^2)$
	req-	can	over	$\Pr(\theta)]$	ϵ
	ui-	sur-	the	$\&\&$	con-
	sites	vive	pa-	Similar	trols
		in	ram-	to	how

EA_RL_parallel.png

e-3e-3\$\$ [t]9.473684210526316e-3 A simple parallel

$$\begin{aligned} & \nabla_{\theta} \mathbb{E}_{\epsilon} \int_0^{\infty} \gamma^t \mathcal{L}(\theta) dt \\ & \sim \mathcal{N}(0, \int_0^{\infty} \gamma^t \sigma^2 dt) \\ & = \frac{1}{\mathbb{E}_{\epsilon} \int_0^{\infty} \gamma^t \sigma^2 dt} \int_0^{\infty} \gamma^t \sigma^2 dt \end{aligned}$$

evolution-strategies-based RL algorithm. Parallel workers share the random seeds so that they can reconstruct the Gaussian noises with tiny communication bandwidth. (Image

e-3e-3

4.2 Deadly Triad Is-sue#

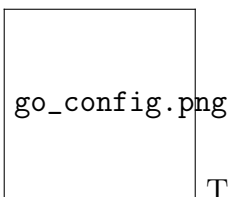
e-3e-3We

do seek the ef-fi-ciency and flex-i-bil-ity of TD meth-ods that in-volve boot-strap-ping. How-ever, when off-policy, non-lin-ear func-tion ap-prox-i-ma-tion, and boot-strap-ping are com-bined in one RL al-go-rithm, the train-ing

28

e-3e-3

5 Case Study: AlphaGo Zero#



e-3e-3The [t]9.473684210526316e-3 The board of Go. Two players

game
of
Go
has
been
an
ex-
tremely
hard
prob-
lem
in
the
field
of
Ar-
ti-
fi-
cial
In-
tel-
li-
gence
for
decades
un-
til
re-
cent
years.
Al-
phaGo
and
Al-
phaGo
Zero
are
two
pro-
grams
de-
vel-
oped
by
a
team
at
Deep-

play black and white stones alternatively on the vacant intersections of a board with 19 x 19 lines. A group of stones must have at least one open point (an intersection, called a "liberty") to remain on the board and must have at least two or more enclosed liberties

(called "eyes") to stay "alive". No stone shall repeat a previous position.

e-3e-3With
all
the
knowl-
edge
of
RL
above,
let's
take
a
look
at
how
Al-
phaGo
Zero
works.
The
main
com-
po-
nent
is
a
deep
CNN
over
the
game
board
con-
fig-
u-
ra-
tion
(pre-
cisely,
a
ResNet
with
batch
nor-
mal-
iza-
tion
and
ReLU).
This
net-
work
out-
puts

alphago-zero-selfplay.p

e^{-3e-3} e^{-3e-3} • e^{-3e-3} During [t]9.473684210526316e-3
 (p, π : self-
 v) the play,
 = game MCTS
 $f_{\theta}(s)$ board fur-
 π con- ther
 fig- im-
 u- proves
 ra- the
 tion, ac-
 19 tion
 x prob-
 19 a-
 x bil-
 17 ity
 stacked dis-
 fea- tri-
 ture bu-
 planes; tion
 17 π
 fea- $\sim$
 tures $p(\cdot)$
 for and
 each then
 po- the
 si- ac-
 tion, tion
 8 a_t
 past is
 con- sam-
 fig- pled
 u- from
 ra- this
 tions im-
 (in- proved
 clud- pol-
 ing icy.
 cur- The
 rent) re-
 for ward
 the z_t
 cur- is
 rent a
 player bi-
 + nary
 8 value
 past $\frac{1}{32}$
 con- in-
 fig- di-
 cat-

AlphaGo Zero is trained by self-play while MCTS improves the output policy further in every step. (Image source: Figure 1a in Silver et al., 2017).

$\mathcal{L}$
 is
 trained
 with
 the
 sam-
 ples
 in
 the
 re-
 play
 mem-
 ory
 to
 min-
 i-
 mize
 the
 loss:

$=$
 $(z$
 $-$
 $v)^2$
 $-$
 π^{top}
 $\log$
 p
 $+$
 c
 $\|$
 θ
 $\|^2$
 $\$$

is
 a
 hy-
 per-
 pa-
 ram-
 e-
 ter
 con-
 trol-
 ling
 the
 in-
 ten-
 sity
 of
 L2
 penalty
 to
 avoid
 over-
 fit-
 ting.

e-3e-3AlphaGo I e-3e-3_ e-3e-3CitedBe-30art3e-3{weng2018band-3[2] e-3e-3[3] e-3e-3[4]
 Zero know as: title 6 = **References** // Vol. 5 (Reinforcement Learning)
 sim- this author = "Weng, Lilian", Mnih, Sal-
 pli- is journal = "Lilian Weng's Github", i-
 fied a year = "2018", ton al. mans,
 Al- long url = "https://lilianweng.github.io/posts/2018-03-03-
 phaGo read, } force- An- chronous al.
 by but drew meth- Evo-
 re- hope- G. ods lu-
 mov- fully Barto. for tion
 ing worth al. Re- deep strate-
 su- it. overviewin- re- gies
 per- *If* arXiv force- in- as
 vised *you* preprint ment force- a
 learn- *no-* arXiv:1701.06724. ment scal-
 ing *tice* 2017. ing: learn- able
 and *mis-* An ing. al-
 merg- *takes* In- ICML. ter-
 ing *and* 2016. na-
 sep- *er-* duc- tion; to
 a- *rors* tion; 2nd re-
 rated *in* Edi- in-
 pol- *this* tion. force-
 icy *post,* 2017. ment
 and *don't* learn-
 value *hes-* ing.
 net- *i-* arXiv
 works *tate* preprint
 into *to* arXiv:1703.0386
 one. *con-* (2017).
 It *tact*
 turns *me*
 out *at*
 that *[lil-*
 Al- *ian*
 phaGo *dot*
 Zero *weng-*
 achieved *weng*
 largely *at*
 im- *gmail*
 proved *dot*
 per- *com*].
 for- See
 mance you
 with in
 a the
 much next
 shorter post!
 train- :)
 ing
 time!
 I
 strongly

e-3e-3[5]	e-3e-3[6]	e-3e-3[7]	e-3e-3[8]	e-3e-3[9]	e-3e-3[10]	e-3e-3[11]	e-3e-3[12]	e-3e-3[13]	e-3e-3[14]
David Silver, et al. Mastering the game of go with-out hu-man knowl-edge. Na-ture (2017): 354.	David Silver, et al. Mastering the game of Go with deep neu-ral net-works and tree search. Na-ture (2016): 484-489.	Volodymyr Mnih, et al. Human-level control through deep learning. Na-ture (2015): 518.7540	Ziyu Wang, et al. Dueling network architectures for deep reinforcement learning. ICML. 2016.	Reinhold Schaefer, et al. Reinforcement Learning with Self-Play. arXiv preprint arXiv:1606.02647. 2016.	OpenAI Blog: Evolution of Strategic Play as a Scalable Algorithm to Reinforce Learning. OpenAI Blog. 2017.	Frank Sehnke, et al. Parameter-free neural networks. Neural Networks. 2013.	Csaba Szepesvári. Al-gorithms for reinforcement learning with function approximation. In: Handbook of reinforcement learning. Springer, 2008.		If you notice mistakes and errors in this post, please don't hesitate to contact me at [lilian.weng@gmail.com] and I would be super happy to correct them right away!

e-3e-3	•	e-3e-3	e-3e-3	e-3e-3©	e-3e-3
		Reinforcement-Policy Learning	Gradient Algorithms	2025 Lil'Log Powered by Hugo & Paper-Mod	
	•	Long-Read			
	•	Math-Heavy	The Multi-Armed Bandit Problem and Its Solutions		

7 Citation

Please cite this work as:

Weng, Lilian. “Title”. Lil 'Log (May 2025). <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>

Or use the BibTeX citation:

```
@article{weng2025,
  title = {},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2025}, month = {May},
  url = {}
}
```

8 References

Tags: [tag1](#) [tag2](#)

[⌵](#) [Title](#) [Title](#) [⌶](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)